Multi-Tenant E-Commerce Platform — Simplified Plan

Version: 2.0 (Simplified) · **Date:** July 27, 2026 **Reference storefront we're matching:** The Meat Hub (themeathub.com) — clean, mobile-first, banner + category tiles + product rows + newsletter + footer.

---

1. The Idea in One Paragraph

One codebase, one server, one database. We add a **company** in our admin, fill in its products, categories, banners and settings, point its domain at our server, and its store is live. Every company's site runs from the same application, but each sees only its own data. The admin panel is **for us and our clients only** — visitors never see it, and clients can never create their own store (only we do that).

Think Shopify, but small, internal, and built exactly for the kind of store The Meat Hub is.

---

2. What We're Building (and What We're Not)

In scope

- Multi-tenant: many companies, one app, data isolated by `COMPANY_ID`.
- Domain-based routing: `zoloft.com` → loads Zoloft's store automatically.
- **Products** with images, price, sale price, stock, SKU, variants (e.g. weight/size), tags.
- **Categories** and **collections**.
- **Homepage & pages** built from **reorderable sections** (toggle on/off + drag to reorder; each section's content is filled from the admin — not free-form drag-and-drop).
- **Banners** (hero slider + promo banners).
- **Menus** (header/footer navigation).
- **Cart + orders**: visitor adds to cart and places an order; the order is **saved and shown to the client admin** (Cash-on-Delivery / manual — **no online payment processing**).
- **Customers**: guest checkout + optional account with order history.
- **CMS pages**: About, Contact, FAQ, Privacy, etc.
- **Media library**: upload images, reuse them.
- **Multi-language**: each company enables its languages; content is translatable; visitor language switcher; untranslated text falls back to the default language.
- **Store settings**: logo, colors, currency label, contact info, social links, SEO basics.
- **Themes**: a company picks a theme; theme = colors/fonts/layout style, not a separate codebase.
- **Mobile-friendly** storefront (non-negotiable — most traffic is mobile).

- **Super Admin** (us): create/manage companies, assign domains, suspend/activate, monitor.

Out of scope (deliberately cut for simplicity)

- ❌ Online payment gateways (Stripe/PayPal) — orders are manual/COD for now.
- ❌ Shipping engine / courier integrations.
- ❌ Multi-currency conversion (each store shows **one** currency; switching added later if needed).
- ❌ Full free-form drag-and-drop page builder (reorderable fixed sections instead).
- ❌ Billing clients / subscriptions.
- ❌ Three separate "engines" / microservices — **one clean codebase with organized folders** is simpler and perfectly fine at this scale.
- ❌ Mobile apps, marketplace, AI generators — future ideas, not now.

Anything cut is cut *cleanly* — the data model leaves room to add payments, shipping, or currency later without a rewrite.

---

3. Tech Stack

Layer	Choice	Note
Backend	Node.js + Express, **plain JavaScript (no TypeScript)**	REST API, JWT auth
DB driver	`node-oracledb`	with a connection pool
Query layer	Knex (query builder)	keeps SQL organized; Sequelize is the alternative
Database	**Oracle 19c Enterprise Edition**	single source of truth
Storefront	Next.js (React), server-side rendered	SSR needed for SEO + fast mobile loads
Admin panels	React single-page app	Tailwind for styling; a component kit like ShadCN is optional
Cache/sessions	Redis	tenant lookup, sessions, page cache
Media	Disk on our server + CDN (Cloudflare) in front	CDN offloads image bandwidth
Proxy/SSL	Nginx	terminates SSL, forwards to the app

---

4. How a Request Works

```
Visitor opens  www.zoloft.com
        ↓
Nginx  (SSL)  →  Node/Express app
        ↓
Tenant Resolver: look up the domain → get COMPANY_ID  (cached in Redis)
        ↓
If company suspended → show maintenance page.  If domain unknown → 404.
        ↓
Set company context on the DB connection  (Oracle VPD then auto-filters every
query)
        ↓
Load that company's theme, menus, homepage sections, products
        ↓
Render the store  (Next.js, mobile-first)
```

The visitor never knows other stores share the same app.

---

5. Keeping Companies' Data Separate (the one thing we must get right)

Every company-owned table has a `COMPANY_ID` column. Isolation is enforced in **three layers**, so a single mistake can't leak data:

1. **Oracle VPD (main protection, database-level):** we set the current company on the DB connection each request, and Oracle automatically adds `COMPANY_ID = <current>` to every query behind the scenes. Even a buggy query can't see another company's rows. *(This is a built-in feature of your Enterprise Edition — no extra cost.)*
2. **Repository layer (app-level):** all database access goes through shared data-access functions that also add the company filter. No loose SQL scattered around the code.
3. **Automated leak tests:** before any release, tests log in as Company A and try to read Company B's data. If anything leaks, the build fails.

One important detail: because DB connections are reused from a pool, we **set the company on borrow and clear it on return** — a small wrapper handles this so context never bleeds between requests. Our own Super Admin uses a separate DB account allowed to see across companies (for management screens only, and audited).

---

6. Database — Main Tables

Platform-level (no `COMPANY_ID`): `COMPANIES`, `DOMAINS` (domain → company), `PLATFORM_USERS` (us / super admins), `THEMES`, `AUDIT_LOG`.

Company-level (all have `COMPANY_ID`):

- **Catalog:** `PRODUCTS`, `PRODUCT_VARIANTS` (sku, price, sale_price, stock, option values), `PRODUCT_OPTIONS`, `PRODUCT_IMAGES`, `CATEGORIES` (with parent for subcategories), `PRODUCT_CATEGORIES`, `COLLECTIONS`, `COLLECTION_PRODUCTS`.
- **Content:** `PAGES` (a page = an ordered list of sections), `PAGE_SECTIONS` (type, position, enabled flag, settings JSON), `BANNERS`, `MENUS` + `MENU_ITEMS`, `MEDIA`.
- **Orders & customers:** `CUSTOMERS`, `CUSTOMER_ADDRESSES`, `CARTS` + `CART_ITEMS`, `ORDERS` + `ORDER_ITEMS` (prices copied in at order time), `ORDER_STATUS_HISTORY`. *(No payment tables — an order just records what was ordered, by whom, and its status: New → Confirmed → Delivered → Cancelled.)*
- **Localization:** `COMPANY_LANGUAGES` (enabled languages + default), plus translation tables per translatable thing (`PRODUCT_TRANSLATIONS`, `CATEGORY_TRANSLATIONS`, `PAGE_TRANSLATIONS`, etc.). Missing translation → falls back to the default language.
- **Settings & staff:** `STORE_SETTINGS` (logo, colors, currency label, contact, social, SEO defaults), `ADMIN_USERS` (client staff), `ROLES`.

Notes: numeric IDs via Oracle `IDENTITY`; JSON stored in CLOB with `IS JSON` checks (used for section settings); "slug is unique **per company**," not globally.

---

7. Homepage / Pages — the Reorderable Section System

A page is just an **ordered list of sections**. The admin can **toggle each section on/off** and **drag to reorder** them. Each section type has its own little settings form. That's it — no free-form canvas, which is exactly why it stays simple while still producing a Meat-Hub-style page.

Section types to build (matches The Meat Hub):

- Hero banner slider (choose images + links)
- Featured categories (pick categories to show as tiles)
- Product row / carousel (pick a collection or hand-pick products)
- Promo banner (single image + "Shop Now" button)
- Best sellers / new arrivals (auto from data)
- Blog/recipe strip (optional)
- Feature icons ("Fast Delivery", "Best Quality"…)
- Newsletter signup
- Rich text (for CMS pages like About)
- Footer (menus + info)

One React "section renderer" is shared by both the live storefront and the admin preview, so what the client arranges is exactly what visitors see.

Themes: a theme is a set of colors, fonts and spacing (design tokens) plus header/footer style. Switching themes changes only appearance, never data — so a company can restyle without losing anything.

---

8. The Three Screens

Storefront (what visitors see)

Mobile-first. Home (sections), category & collection pages with sorting/filtering, product page (image gallery, variant picker, add to cart), search, cart, simple checkout that **saves the order** (name, phone, address, COD), order confirmation, optional customer login + order history, CMS pages, language switcher, working SEO (titles, meta, sitemap, hreflang for languages).

Client Admin (what each company manages — no developer needed)

Dashboard (orders + sales summary), Products (add/edit/delete, images, stock, price/sale, variants), Categories & collections, Orders (view, change status, mark delivered/cancelled), Customers, Banners, Pages (arrange sections), Menus, Media library, Translations, Store settings (logo, colors, currency, contact, SEO), Staff & roles.

Super Admin (only us)

Create a company (one screen → auto-creates admin user, default homepage, default categories, default menus, default settings so the store is instantly usable), manage all companies, assign domains, suspend/activate (suspended = maintenance page), manage themes, monitor all stores (orders/day, storage), "view as company" for support (audited).

---

9. Creating a New Company (the whole point)

One form: name, domain, email, phone, logo, theme, currency, default language, timezone, status. On **Create**, the system auto-generates: the company record, an admin login, a default homepage (with sensible sections), a few starter categories, default menus, default SEO, and default settings. The client logs in and starts adding products. **No coding, no deploy.** (Pointing the domain's DNS to our server is the one manual step we do.)

---

10. Build Order (roughly 3–4 months, small team)

Phase 0 — Foundation (3 weeks). Project setup, Oracle schema, tenant resolver + Redis, VPD + connection wrapper + leak tests, login/roles, media upload, Super Admin "create company." *Done when: two companies on two local domains show different stores with proven data isolation.*

Phase 1 — Catalog + Storefront shell (4–5 weeks). Products (variants, images, stock), categories, collections, banners, menus, settings. Mobile-first storefront: home, category,

product, search. Section system v1 (toggle + reorder). Multi-language foundation (languages, translations, `/en/`, `/ar/` routing, translations editor).

Phase 2 — Cart & Orders (3–4 weeks). Cart, checkout that saves the order (COD/manual), order management for the client, order-status emails, customer accounts + guest checkout, client dashboard.

Phase 3 — Polish & Launch (2–3 weeks). SEO pass, mobile QA, caching/performance, Super Admin monitoring + suspend/activate, backups + runbooks (how to add a domain, how to onboard a company), pilot with one real client.

Later (only if wanted): online payments (Stripe/PayPal — the order model is already ready for it), shipping rates, currency switching, full drag-and-drop builder, reviews, WhatsApp order button.

---

11. Main Risks (short)

- **Data leak between companies** → the 3-layer isolation in §5; leak tests block releases.
- **Scope creeping back up** → payments/shipping/currency stay out until v1 ships; the schema leaves room so adding them later isn't a rewrite.
- **Mobile performance** (most visitors) → SSR + CDN for images + caching; test on real phones each phase.
- **One server = single point of failure** → keep the app stateless (sessions in Redis) so a second server can be added; tested Oracle backups.

---

12. Open Items (safe defaults assumed — correct if wrong)

- Product **variants** (e.g. weight/size like Meat Hub's "From 29 SR") → assumed **yes**.
- **Team size** → assumed 2–4 developers (affects timeline).
- **Customer accounts** → assumed guest checkout **plus** optional registration.

13. Suggested Next Deliverable

The **Oracle schema (DDL)** — every table above, keys, indexes, the VPD policy, and the company-context procedure, ready to run on your 19c server. Everything else builds on it.